Problem Statement

Hospital Staff Management System

Background:

A hospital wants to digitize its staff management process. The management needs a backend application to manage hospital staff information and secure access to the system.

The application should be developed using:

- FastAPI
- PostgreSQL

And Demonstrate

- Pagination APIs
- Search APIs
- Filtering APIs
- User Registration APIs
- Login APIs
- JWT Authentication
- Password Hashing
- Role-Based Access Control (RBAC)

Database Details

Database Name: **hospital_db**

Table Name: **HospitalStaff**

Columns

Column Name	Data Type
id	Integer
username	String
password	String
role	String
department	String

Sample Roles

• Admin
• Doctor
• Receptionist

Tasks to Perform

Task 1: Create PostgreSQL database.

Task 2: Create:
- database.py
- models.py
- schemas.py
- main.py

Task 3: Create SQLAlchemy model.

Task 4: Create Pydantic Models for:
- User Registration
- User Login

Task 5: Create User Registration API

Requirements:
- Register new staff members
- Store username, password, role and department

Task 6: Implement Password Hashing

Requirements:
- Hash password before storing it in the database
- Use bcrypt hashing

Task 7: Create Login API

Requirements:
- Verify username
- Verify password
- Generate JWT token after successful login

Task 8: Implement JWT Authentication

Requirements:
- Create JWT token generation function
- Create JWT verification function

Task 9: Create GET API with Pagination

Requirements: Fetch staff records using:
- skip

• limit

Example:

/staff?skip=0&limit=5

Task 10: Create Search API

Requirements: Search staff records by: • username

Example:

/search?username=amit

Task 11: Create Filtering API

Requirements: Filter staff records by: • department

Example:

/filter?department=Cardiology

Task 12: Implement Role-Based Access Control (RBAC)

Requirements: Only Admin can: • Add Staff

• Update Staff

• Delete Staff

Doctor & Receptionist can: • View Staff Records

Task 13: Create Admin-Only API

Requirements: Protect at least one API using role verification.

Example: • Add Staff OR

• Delete Staff

Task 14: Testing

Demonstrate: • User Registration

• Login • JWT Token Generation

• Password Hashing • Search API

• Filtering API

• Pagination API

• Role-Based Access Control

Expected Outcome

Students should demonstrate the ability to:

• Build Registration APIs

- Build Login APIs
- Implement JWT Authentication
- Implement Password Hashing
- Implement Search APIs
- Implement Filtering APIs
- Implement Pagination APIs
- Implement Role-Based Access Control using FastAPI and PostgreSQL